

# SSR in Angular:

## ➤ Do You Really Need It?

Learn when SSR is the right choice—and when it just adds unnecessary complexity.



**Sreeram Saikumar**

Angular Developer

 /sreeramsaikumar

## Server-Side Rendering:

It's a technique where the HTML of your Angular application is generated on the server and then sent to the browser, instead of the browser generating the HTML after loading the JavaScript (which is how most Angular apps work by default).

 /sreeramsaikumar

## Why use SSR?

- **Faster Page Load:** Since the browser gets a fully-rendered page from the server, the user can see the content immediately, without waiting for all the JavaScript to be processed and rendered.
- **SEO (Search Engine Optimization):** Search engines can read the content of your page directly because the HTML is already generated when it arrives at the browser. Without SSR, search engines may not be able to properly crawl JavaScript-heavy pages.
- **Improved Performance:** It can improve the perceived performance, as the initial content loads faster.

 /sreeramsaikumar

## How SSR Works in Angular?

- **Client-Side Rendering (CSR):** In a normal Angular app (without SSR), the browser downloads the Angular app (a JavaScript bundle), and the JavaScript renders the page.
- **Server-Side Rendering (SSR):** In Angular with SSR, the initial request (from a browser) is sent to the server. The server runs Angular code, generates the HTML (renders the page), and sends it back to the browser. The browser then shows the rendered page instantly. After that, the Angular app "takes over" on the client side, like normal.

 /sreeramsaikumar

## Angular and SSR: Angular Universal

To enable SSR in Angular, Angular provides a technology called **Angular Universal**. It's a set of tools and libraries that allow you to render Angular apps on the server.

 /sreeramsaikumar

# How to Set Up SSR in Angular (Angular Universal)?

Here's a simple step-by-step process:

**1. Create an Angular App** (if you don't have one yet):

```
ng new my-angular-app
```

**2. Add Angular Universal** (this will set up SSR for you):

```
ng add @nguniversal/express-engine
```

This command adds the necessary dependencies and modifies your Angular app to support SSR with Express, which is a web server for Node.js.

 /sreeramsaikumar

### 3. Build the Universal App:

Once set up, you'll have two build targets:

- **browser:** Regular Angular build (client-side)
- **server:** Server-side rendering build

To build the app for SSR:

```
ng build --prod
```

```
ng run my-angular-app:server:production
```

### 5. Run the Universal App:

After building, run it using Node.js:

```
node dist/my-angular-app/server/main.js
```

This will start your app on a local server. When you visit it in the browser, the first page you see is already rendered HTML from the server!

 /sreeramsaikumar

## What Happens Behind the Scenes?

1. When a request comes in, the server executes Angular's code and renders the HTML.
2. The HTML is sent to the browser.
3. After the initial load, Angular's client-side code takes over, and the app behaves like a normal single-page app (SPA), meaning further page changes are handled on the client side.

 /sreeramsaikumar

# When to Use SSR

## 1. SEO Optimization

- **Use case:** Public websites where SEO matters.
- **Why:** Helps search engines crawl and index content easily.

## 2. Faster Page Loads

- **Use case:** Content-heavy pages like blogs or e-commerce sites.
- **Why:** Pre-renders HTML, so users see content faster.

 /sreeramsaikumar

## When to Use SSR (Cont'd)

### 3. Improved Performance on First Load

- **Use case:** Apps with heavy initial content (like articles or product listings).
- **Why:** Reduces perceived load time by showing pre-rendered HTML.

### 4. Accessibility

- **Use case:** Pages that need to be visible to users even before full JavaScript loads.
- **Why:** Makes your content accessible immediately.

 /sreeramsaikumar

# When Not to Use SSR

## 1. No SEO Requirement

- **Use case:** Internal apps (dashboards, admin panels).
- **Why:** If SEO isn't a priority, SSR isn't worth the complexity.

## 2. Highly Dynamic Content

- **Use case:** Apps with personalized content (social media feeds, live data).
- **Why:** SSR works best with static content, not content that changes often.

 /sreeramsaikumar

## When Not to Use SSR (Cont'd)

### 3. High Server Load or Cost

- **Use case:** High-traffic sites with complex logic (e-commerce).
- **Why:** SSR increases server load, which could lead to scalability issues.

### 4. Real-Time Apps

- **Use case:** Chat apps, collaborative apps, real-time games.
- **Why:** Real-time updates are more efficient with Client-Side Rendering (CSR).

 /sreeramsaikumar

## When Not to Use SSR (Cont'd)

### 5. Heavy JavaScript-Dependent Content

- **Use case:** Apps with complex interactivity and animations.
- **Why:** SSR is better for static content; CSR handles dynamic UIs better.

### 6. Limited Resources for Optimization

- **Use case:** Small teams, quick prototypes, MVPs.
- **Why:** SSR setup and optimization can be time-consuming.

 /sreeramsaikumar

## Ready to try SSR in Angular?

- Share your thoughts or drop a question in the comments below!
- Let's chat about Angular and web performance.

 /sreeramsaikumar

 /sreeramsaikumar

+ Follow

 /sreeramsaikumar